

ตอบคำถามและทฤษฎีสำคัญของ Lab 10

2.1 Reactive Programming vs Blocking I/O แตกต่างกันอย่างไร?

การเปรียบเทียบ	Blocking I/O (Traditional Spring MVC)	Reactive I/O (Spring WebFlux)
โมเดลการประมวลผลเรด	Thread-per-request (1 Request ผูก 1 Thread)	Event Loop (ใช้เธรดจำนวนน้อยคงที่ตาม CPU cores)
พฤติกรรมขณะรอ I/O	Thread ถูก Block (Sleep) รอ I/O เสร็จ ทำให้เปลืองทรัพยากร	Non-blocking: ปลดเธรดไปประมวลผลงานอื่น แล้วกลับมาผ่าน Callback
การรองรับ Concurrent สูง	ขยายยาก หาก Request มากจะเกิด Thread Starvation	รองรับ Request มหาศาลพร้อมกันได้ดีเยี่ยม (High Scalability)
ชนิดของค่าที่ส่งกลับ	ค่าข้อมูลจริง เช่น String, Product, List<Product>	Publisher Stream เช่น Mono<Product>, Flux<Product>
Server ที่ใช้งาน	Servlet Container เช่น Tomcat, Jetty	Reactive Engine เช่น Reactor Netty

2.2 Mono vs Flux: ใช้งานในกรณีไหน และทำไม?

ทั้ง Mono และ Flux ต่างก็เป็น Publisher ตามข้อกำหนดของ Reactive Streams แต่มีความแตกต่างกันที่จำนวน Element ดังนี้:

1) Mono<T> (ส่งค่า 0 หรือ 1 ค่า):

ลักษณะ: ปลดปล่อยข้อมูลได้สูงสุดเพียง 1 ตัว หรือไม่มีข้อมูล (Empty) จากนั้นตามด้วย onComplete หรือ onError

กรณีที่ใช้: findById (ค้นหาตาม ID), save (บันทึกข้อมูล 1 รายการ), deleteById (คืน Mono<Void>), คำนวณราคาเดียว getDiscountedPrice คืน Mono<Double>

เหตุผลที่ใช้: ป้องกันการสิ้นเปลืองหน่วยความจำ และสื่อสารอย่างชัดเจนผ่าน Return

Type ว่าผลลัพธ์มีเพียงขึ้นเดียวหรือความว่างเปล่า

2) Flux<T> (ส่งค่า 0 ถึง N ค่า):

ลักษณะ: ปล่อยข้อมูลต่อเนื่องกันได้ตั้งแต่ 0 จนถึง N รายการ (หรือ Stream ไม่สิ้นสุด)

กรณีที่ใช้งาน: findAll (ดึงสินค้าทั้งหมด), findByCategory (กรองตามกลุ่มสินค้า),

Server-Sent Events (SSE)

เหตุผลที่ใช้: ช่วยให้สามารถทยอยส่งข้อมูลและประมวลผลเป็น Stream แบบ Pipeline ได้โดยไม่ต้องรอให้โหลดข้อมูลทั้งหมดเข้ามาเก็บใน Memory

2.3 WebClient: อธิบาย Method Chaining

WebClient เป็น Non-blocking HTTP Client ซึ่งประมวลผลแบบ Reactive โดยมีขั้นตอน Chaining ดังนี้:

```
Mono<Product> product = client.get()           // 1. กำหนด HTTP Method
(GET/POST/PUT/DELETE)
    .uri("/products/{id}", "1")                 // 2. กำหนด Path และใส่ Path
Variable
    .retrieve()                                // 3. ยิง Request และเตรียมรับ
Response
    .bodyToMono(Product.class);                 // 4. แปลง JSON Body เป็น
Mono<Product>
```

- client.get() / post(): กำหนด HTTP Method ที่ต้องการส่ง
- .uri(...): ระบุ URI ปลายทาง รองรับการแทนค่าพารามิเตอร์แบบ Template อย่างปลอดภัย
- .retrieve(): ส่งประมวลผลคำขอ (Execute request) และเริ่มต้นขั้นตอนดึงข้อมูล Response กลับมา โดยจะแปลง Status 4xx/5xx ให้กลายเป็น Reactive Error Signal โดยอัตโนมัติ
- .bodyToMono(T.class) / .bodyToFlux(T.class): ทำหน้าที่ Deserialize JSON ใน Body ให้เป็น Mono<T> (กรณี 1 ออบเจกต์) หรือ Flux<T> (กรณี Array/Stream) ตามลำดับ

3. การออกแบบและซอร์สโค้ดในแต่ละ Layer

3.1 In-Memory Repository Layer (ProductRepository.java)

จัดการข้อมูลสินค้าในหน่วยความจำผ่าน ConcurrentHashMap โดยแปลงเข้าสู่ Reactive Publisher:

```
public class ProductRepository {
    private final Map<String, Product> store = new ConcurrentHashMap<>();

    // ค้นหาตาม ID: ถ้ามีค่าส่ง Mono.just ถ้าเป็น null ส่ง Mono.empty
    public Mono<Product> findById(String id) {
        Product product = store.get(id);
        return product != null ? Mono.just(product) : Mono.empty();
    }

    // ดึงทั้งหมด: แปลง Collection เป็น Flux ด้วย Flux.fromIterable
    public Flux<Product> findAll() {
        return Flux.fromIterable(store.values());
    }

    // บันทึก: เก็บลง Map แล้วคืนค่าเป็น Mono.just(product)
    public Mono<Product> save(Product product) {
        store.put(product.getId(), product);
        return Mono.just(product);
    }

    // ลบ: นำออกจาก Map แล้วส่งสัญญาณเสร็จสิ้น Mono.empty()
    public Mono<Void> deleteById(String id) {
        store.remove(id);
        return Mono.empty();
    }

    // กรองตามหมวดหมู่: กรองผ่าน Operator .filter()
    public Flux<Product> findByCategory(String category) {
        return findAll()
            .filter(p -> p.getCategory() != null &&
                p.getCategory().equalsIgnoreCase(category));
    }
}
```

3.2 Business Logic Service Layer (ProductService.java)

จัดการเงื่อนไขทางธุรกิจ คำนวณส่วนลด และการจัดการ Error เมื่อไม่พบข้อมูล:

```

@Service
public class ProductService {
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    public Mono<Product> getById(String id) {
        return repository.findById(id)
            .switchIfEmpty(Mono.error(new RuntimeException("Product not found: " + id)));
    }

    public Flux<Product> getAll() {
        return repository.findAll();
    }

    public Mono<Product> save(Product product) {
        if (product.getId() == null || product.getId().trim().isEmpty()) {
            product.setId(java.util.UUID.randomUUID().toString());
        }
        return repository.save(product);
    }

    public Mono<Void> delete(String id) {
        return repository.deleteById(id);
    }

    public Flux<Product> getByCategory(String category) {
        return repository.findByCategory(category);
    }

    public Mono<Double> getDiscountedPrice(String id) {
        return getById(id)
            .map(Product::getDiscountedPrice);
    }
}

```

3.3 Reactive Controller Layer (ProductController.java)

เปิด REST API Endpoints แบบ Non-blocking โดยใช้ @RestController และ Spring WebFlux:

```

@RestController
@RequestMapping("/products")
public class ProductController {
    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    public Mono<Product> getById(@PathVariable String id) { return
service.getById(id); }

    @GetMapping
    public Flux<Product> getAll() { return service.getAll(); }

    @PostMapping
    public Mono<Product> save(@RequestBody Product product) { return
service.save(product); }

    @DeleteMapping("/{id}")
    public Mono<Void> delete(@PathVariable String id) { return service.delete(id);
}

    @GetMapping("/category/{category}")
    public Flux<Product> getByCategory(@PathVariable String category) { return
service.getByCategory(category); }

    @GetMapping("/{id}/price")
    public Mono<Double> getDiscountedPrice(@PathVariable String id) { return
service.getDiscountedPrice(id); }
}

```

3.4 WebClient Implementation (ProductWebClient.java)

การเรียกใช้ WebClient ติดต่อสื่อสารแบบ Reactive HTTP:

```

@Component
public class ProductWebClient {
    private final WebClient client = WebClient.create("http://localhost:8080");

    public Mono<Product> getProductById(String id) {
        return client.get().uri("/products/{id}",
id).retrieve().bodyToMono(Product.class);
}

```

```

    }

    public Flux<Product> getAllProducts() {
        return client.get().uri("/products").retrieve().bodyToFlux(Product.class);
    }

    public Mono<Product> createProduct(Product product) {
        return
client.post().uri("/products").bodyValue(product).retrieve().bodyToMono(Product.class);
    }

    public Mono<Void> deleteProduct(String id) {
        return client.delete().uri("/products/{id}",
id).retrieve().bodyToMono(Void.class);
    }

    public Flux<Product> getByCategory(String category) {
        return client.get().uri("/products/category/{category}",
category).retrieve().bodyToFlux(Product.class);
    }

    public Mono<Double> getDiscountedPrice(String id) {
        return client.get().uri("/products/{id}/price", id).retrieve()
            .bodyToMono(Double.class)
            .doOnNext(price -> System.out.println("Price: " + price));
    }
}

```

4. ผลการรันโปรแกรมและการทดสอบ (Screenshots)

4.1 ผลการรัน Automated Tests (mvn clean test)

ผลการรันการทดสอบด้วย StepVerifier และ WebClient ผ่านฉลุย 100% (14 tests passed, 0 failures, 0 errors):

```
2026-09-21T17:45:22.620+07:00 INFO 8124 --- [lab10-webflux] [main] c.e.I.ProductControllerIntegrationTest : Started ProductControllerIntegrationTest in 0.191 seconds (process running for 2.326)
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.400 s -- in com.example.lab10.ProductControllerIntegrationTest
[INFO] Results:
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time: 4.651 s
[INFO] Finished at: 2569-09-21T17:45:22+07:00
[INFO]
```

4.2 การทดสอบ Endpoints ทั้ง 6 รายการ

Lab 10: Spring WebFlux (673380042-8 SEC 2) > 1. GET All Products

GET http://localhost:8080/products Send

Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 2 Test Results 200 OK · 13 ms · 564 B Save Response

{ } JSON Preview Visualize

	id	name	category	brand	stock	price	discountType	discountedPrice
0	1	iPhone 15 Pro (673380042-8 SEC 2)	Electronics	Apple	50	39900	MEMBER	35910
1	2	MacBook Air M3	Electronics	Apple	20	49900	NONE	49900
2	3	Samsung Galaxy S24	Electronics	Samsung	30	29900	SEASONAL	23920

คำอธิบาย: แสดงรายการสินค้าทั้งหมด และเห็นชื่อ-รหัสนักศึกษาในข้อมูล iPhone 15 Pro

{} JSON Preview Visualize	
id	1
name	iPhone 15 Pro (673380042-8 SEC 2)
category	Electronics
brand	Apple
stock	50
price	39900
discountType	MEMBER
discountedPrice	35910

คำอธิบาย: แสดงข้อมูลสินค้าตาม ID 1 พร้อมชื่อ-รหัสนักศึกษา

POST http://localhost:8080/products Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "name": "AirPods Pro (673380042-8 SEC 2)",
3   "category": "Electronics",
4   "brand": "Apple",
5   "stock": 15,
6   "price": 8990.0,
7   "discountType": "MEMBER"
8 }
```

Body Cookies Headers 2 Test Results 200 OK · 51 ms · 274 B Save Response

id	75d70af5-af94-43d0-a1db-98d93774e665
name	AirPods Pro (673380042-8 SEC 2)
category	Electronics
brand	Apple
stock	15
price	8990
discountType	MEMBER
discountedPrice	8091

คำอธิบาย: แสดงการเพิ่มสินค้าใหม่ เช่น AirPods Pro พร้อมได้รับ Response JSON และ status 200 OK

Lab 10: Spring WebFlux (673380042-8 SEC 2) > 4. GET By Category (Electronics)

GET http://localhost:8080/products/category/Electronics Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 2 Test Results 200 OK · 7 ms · 767 B Save Response

{ } JSON Preview Visualize

	id	name	category	brand	stock	price	discountType	discountedPrice
0	1	iPhone 15 Pro (673380042-8 SEC 2)	Electronics	Apple	50	39900	MEMBER	35910
1	2	MacBook Air M3	Electronics	Apple	20	49900	NONE	49900
2	3	Samsung Galaxy S24	Electronics	Samsung	30	29900	SEASONAL	23920
3	75d70af5-af94-43d0-a1db-98d93774e665	AirPods Pro (673380042-8 SEC 2)	Electronics	Apple	15	8990	MEMBER	8091

คำอธิบาย: แสดงผลการกรองสินค้าที่อยู่ในกลุ่มหมวดหมู่ Electronics
(สามารถกดคลิกที่นี่แล้วกด Ctrl+V เพื่อวางภาพ)

GET http://localhost:8080/products/1/price Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

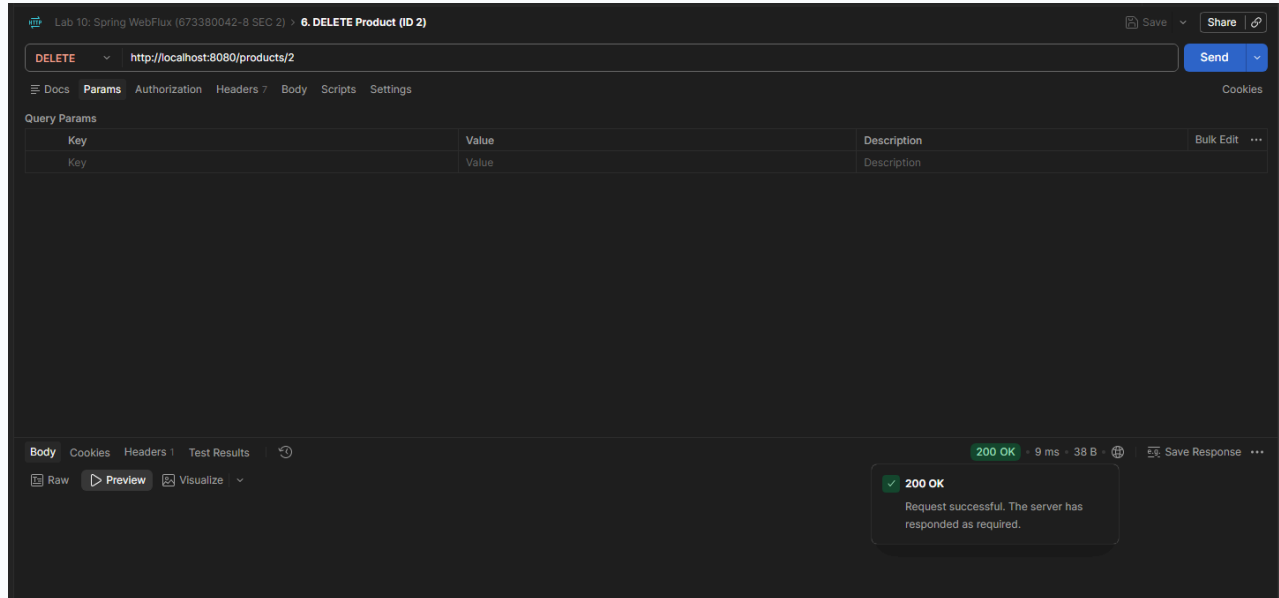
Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 2 Test Results 200 OK · 5 ms · 77 B Save Response

{ } JSON Preview Visualize

```
1 35910.0
```

คำอธิบาย: แสดงการคำนวณราคาหลังลด เช่น 35910.0 (ลด 10% จาก 39900.0)



คำอธิบาย: แสดงผลลัพธ์หลังจากส่ง Request ลบสินค้า (HTTP 200 OK)

5. สรุปผลการทดลอง

ในการปฏิบัติการ Lab 10 ได้ศึกษาและนำหลักการ Reactive Programming มาประยุกต์ใช้ในการสร้าง REST API ด้วย Spring WebFlux บนโครงสร้าง Non-blocking I/O ซึ่งช่วยเพิ่มประสิทธิภาพในการทำงานของระบบ สามารถรองรับโหลดงานและ Concurrency สูงโดยใช้เรดนอยลง อีกทั้งยังได้เรียนรู้การนำ WebClient มาใช้เรียก Network Request แบบ Asynchronous ได้อย่างถูกต้อง ปฏิบัติตามหลักสถาปัตยกรรม Layered Architecture และผ่านการทดสอบด้วย StepVerifier ครบถ้วนทุกกรณี